WEEK 3 · LESSON 13 OF 24 · HANDOUT

Cross-Validation: Testing Without Luck

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=fSytzGwwBVw — open the lesson online to watch inside the course.

🎓 **Go deeper:** Data School: Selecting the best model using cross-validation — https://www.youtube.com/watch?v=6dbrR-WymjI

🎓 **Go deeper:** SciPy 2018 tutorial: Machine Learning with scikit-learn, Part 1 (scikit-learn core developers) — https://www.youtube.com/watch?v=4PXAztQtoTg

Your goal in this lesson: Show why one train/test split can mislead, use k-fold cross-validation to compare models, and keep a final test set untouched.

One split is one roll of the dice

In Week 2 you saw that a model's test score changes when the split changes. Let's measure how much:

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X = loans.drop(columns="defaulted")
y = loans["defaulted"]

scores = []
for seed in range(10):
    X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25,
                                              random_state=seed, stratify=y)
    m = make_pipeline(StandardScaler(), LogisticRegression()).fit(X_tr, y_tr)
    scores.append(m.score(X_te, y_te))
print(np.round(scores, 3))
print("lowest", min(scores), "highest", max(scores))
```

The **same model** scores anywhere from about **0.82 to 0.89** depending only on which borrowers landed in the test set. If you compared two models on one lucky split, you could pick the wrong one.

k-fold cross-validation

Cross-validation uses every row for testing once:

1. Split the training data into **k** equal parts (folds), for example k = 5.
2. Train on 4 folds and test on the 5th.
3. Repeat 5 times, so each fold is the test fold once.
4. Report the **average** score and its **spread** (standard deviation).

Round	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
1	test	train	train	train	train
2	train	test	train	train	train
3	train	train	test	train	train
4	train	train	train	test	train
5	train	train	train	train	test

Three sets, three jobs

Professionals keep a **final test set** locked away and use cross-validation on the rest:

- **Training folds** — the model learns.
- **Validation folds** (inside cross-validation) — you compare models and settings.
- **Test set** — used **once**, at the very end, to report the final score.

```
from sklearn.model_selection import train_test_split

X = loans.drop(columns="defaulted")
y = loans["defaulted"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
```

```
from sklearn.model_selection import cross_val_score

model = make_pipeline(StandardScaler(), LogisticRegression())
cv_scores = cross_val_score(model, X_train, y_train, cv=5)
print(cv_scores.round(3))
print("mean", cv_scores.mean().round(3), "+/-", cv_scores.std().round(3))
```

You should see five scores between about 0.84 and 0.87, with a mean of about **0.86 ± 0.01**. That "± 0.01" is valuable: it tells you how much the score wobbles.

Because the scaler is inside the pipeline, it is re-fitted on the training folds in every round — no leakage.

Compare models fairly

```
from sklearn.model_selection import StratifiedKFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
candidates = {
    "Logistic regression": make_pipeline(StandardScaler(), LogisticRegression()),
    "k-NN (k=15)": make_pipeline(StandardScaler(), KNeighborsClassifier(15)),
    "Random forest": RandomForestClassifier(300, min_samples_leaf=5, random_state=42,
n_jobs=-1),
    "Gradient boosting": GradientBoostingClassifier(random_state=42),
}
for name, m in candidates.items():
    s = cross_val_score(m, X_train, y_train, cv=cv)
    print(f"{name:20s} {s.mean():.3f} +/- {s.std():.3f}")
```

StratifiedKFold keeps the 21% default rate in every fold; `shuffle=True` mixes the rows first. Logistic regression comes out on top (about 0.86), but the others are within about 0.015 — roughly one standard deviation. When scores are that close, prefer the **simpler** model.

Choosing k

- **k = 5 or 10** is standard.
- More folds = more training data per round but more computing time.
- **Leave-one-out** (k = number of rows) is only for tiny datasets.

For data in **time order** (sales per month, exchange rates), never shuffle: use `TimeSeriesSplit`, which always trains on the past and tests on the future.

Key terms

- **Cross-validation** — repeated training and testing on different folds.
- **Fold** — one of the k equal parts of the data.
- **Validation set** — data used to compare models and settings.
- **Stratified k-fold** — folds that keep the class proportions.
- **TimeSeriesSplit** — cross-validation that respects time order.

Remember this: One split is luck; cross-validation gives an average and a spread, and the final test set is opened only once.

Name: _____ Date: _____

Activity

Task 1

Run `cross_val_score` with `cv=10`. Does the mean change? Does the spread change?

Task 2

Use `cross_validate(model, X_train, y_train, cv=5, return_train_score=True)` and compare train and validation scores. Is the model overfitting?

Task 3

Add a `DecisionTreeClassifier()` with no depth limit to the comparison. Where does it rank?

Task 4

Explain to a classmate why you must not tune a model by looking at the test set.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why can a single train/test split be misleading?
   - A. It uses no data
   - B. The score depends on which rows happened to land in the test set
   - C. It always overfits
   - D. It is too slow
2. In 5-fold cross-validation, how many times is the model trained?
   - A. 10
   - B. 5
   - C. 4
   - D. 1
3. What does the ± value after a cross-validation mean tell you?
   - A. How much the score varies between folds
   - B. The learning rate
   - C. The training time
   - D. The number of features
4. When should the final test set be used?
   - A. Many times while tuning
   - B. Before cleaning
   - C. Never
   - D. Once, at the end, to report the final score
5. Why use StratifiedKFold for the loans data?
   - A. To remove defaulters
   - B. To keep the default rate the same in every fold
   - C. To sort by income
   - D. To scale data
6. Which cross-validation suits monthly sales data?
   - A. No validation
   - B. Shuffled KFold
   - C. TimeSeriesSplit
   - D. Leave-one-out with shuffling

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.